



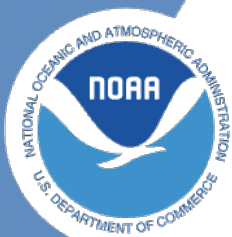
NOAA Technical Memorandum NMFS-XXX-##

My GrandOrgue Implementation

Matt Grossi

January 2023

U.S. DEPARTMENT OF COMMERCE
National Oceanic and Atmospheric
Administration
National Marine Fisheries Service
Northwest Fisheries Science Center



NOAA
FISHERIES

My GrandOrgue Implementation

Making sense of scattered documentation and user experiences

Matt Grossi

Table of contents

Executive Summary	1
1 Hardware	2
1.1 BLUF	2
1.2 Discussion	2
2 Set-up	5
2.1 Raspberry Pi setup	5
2.2 GrandOrgue installation	6
3 Customize	9
3.1 Edit and add your pages	9
3.2 Add your pages the project	9
4 Customization	10
4.1 Bureå Funeral Chapel	10
5 Rendering	12
5.1 Step 1. Make sure you have a recent RStudio	12
5.2 Step 2. Clone and create RStudio project	12
5.3 Step 3. Render within RStudio	12
6 References	13
References	14

List of Figures

List of Tables

1.1	Total cost of my initial proof-of-concept setup.	4
-----	--	---

Executive Summary

This site documents my experience setting up, configuring, and using a GrandOrgue virtual pipe organ (VPO). As of 2026, there was little compiled documentation other than how to download. Information such as system and hardware requirements was scattered throughout the internet, but one generally had to find it through a search engine. The official software documentation consisted of various help pages, but no step-by-step instructions. Lots of community user experiences were captured in hundreds of issues and thousands of discussions on the official GitHub repo. Most of the information here came from those sources.

Getting a functioning VPO took considerable trial and error. The dead ends and failures are not contained here, but important “gotchas” are noted when helpful.

These pages are my contribution to the open source GrandOrgue project and its outstanding user community. I hope these pages help you build your own GrandOrgue VPO.

1 Hardware

Requirements and considerations

1.1 BLUF

The minimum requirements needed to get started with GrandOrgue are:

- Computer with at least 8 GB RAM (16+ GB preferred)
- Touch screen monitor (for interacting with virtual console controls)
- MIDI-USB cable or MIDI cable with a USB audio interface device for connecting the computer to the MIDI instrument
- MIDI-ready organ or keyboard

And, if you don't want the sound to play from your computer speakers:

- Audio cable to port audio from computer to an organ amplifier
or
- Quality external speakers to connect directly to the computer

1.2 Discussion

GrandOrgue is free open source computer software supported for Windows, MacOS, and at least some Linux distributions. System and hardware requirements are summarized here, with the most important requirement being **RAM**.

My initial installation was intended to proof-of-concept learning experiment to assess whether or not it was something I wanted to delve further into and scale up. I already

1 Hardware

had an old (circa 1997) Casio MIDI keyboard¹, so I decided to use that as the workbench test case for experimentation. Then, once I knew what I was doing and felt comfortable with the technical components, I wanted to transfer the setup to a MIDI-ready digital church organ to try it out “in the field” with an actual organ console, decent speakers, and a room where an organ is regularly used.

With this in mind, I sought portability with minimal spatial footprint. Desktop computers took up too much space and are not easily portable, so they were out. Without having a personal organ at home, I needed to interface with the software directly through the computer rather than via an organ console, so I wanted to have a touch screen monitor that could be positioned in close proximity to the music keyboard. While standard laptops with touch screens are portable and could work, I would have to always reach over the laptop keyboard to reach the screen, which was not ideal. My solution was to use a **Raspberry Pi** together with a simple **touch screen monitor**.

I opted for the Raspberry Pi 5 because it was the first and, as of 2026, only unit to support 16 GB of RAM. I purchased a full starter kit for \$385 which included:

- Raspberry Pi 5 with 2.4Ghz 64-bit quad-core CPU and 64 GB of RAM
- 128GB Micro SD Card pre-loaded with 64-bit Raspberry Pi OS, USB MicroSD Card Reader
- Plastic protective case
- Low noise bearing system fan
- Heat sink
- 45W PD Power Supply
- 2 6-foot display cables

I bought a simple and compact 15.6” portable touch screen monitor for \$95 as my primary means of interacting with GrandOrgue. An upgrade might be in store down the road, but this will due to start.

Unfortunately, the Raspberry Pi 5 does not have a dedicated 3.5mm headphone audio jack like its predecessor. Audio is outputted through HDMI or USB instead. I ended up buying a Behringer UMC204HD **USB audio interface** for \$116 which would allow me to use a single USB port for both audio and to connect standard MIDI cables instead of a MIDI-USB cable. I also bought a 9.8-foot dual 5-pin **MIDI cable** for \$11 to go along with that because I did not have one already.

Altogether, my expenses for the initial set up came out to be just over \$600:

¹Old keyboards can almost always be found on Facebook Marketplace or similar sites for free or very little money. Just make sure it supports MIDI, which most manufactured after the 1980s will.

1 Hardware

Table 1.1: Total cost of my initial proof-of-concept setup.

Expense	Cost (USD)
Raspberry Pi 5 kit	\$385
Behringer UMC204HD	\$116
Touch screen monitor	\$95
MIDI cable	\$11
Keyboard	\$0
TOTAL	\$607

Not bad for an authentic-sounding practice organ! The pages that follow describe how I set up, configured, and tested everything.

2 Set-up

Setting up the hardware and software for GrandOrgue

2.1 Raspberry Pi setup

Raspberry Pi (RPi) devices run Debian-based Raspberry Pi OS. GrandOrgue needs the 64-bit Raspberry Pi OS in order to utilize all of the installed RAM (a 32-bit OS would cap the usable RAM at 4 GB, regardless of what is installed in the system.) This comes pre-installed on the CanaKit RPi.

If you don't have the 64-bit version:

1. On another computer, open the RPi Imager and use the gear icon to pre-configure your Wi-Fi credentials, hostname, and enable SSH. This saves time on first boot.
2. Use the Imager on this computer to flash Raspberry Pi OS (64-bit) to your RPi SD card. Be sure to choose to full desktop version.

Back on the RPi, before installing anything, download and install any OS updates:

```
sudo apt update && sudo apt full-upgrade -y  
sudo reboot
```

GrandOrgue recommends X11 to properly handle window sizing and positioning. To switch the display mode:

1. Open a terminal and run `sudo raspi-config`
2. Use the down arrow key to select **Advanced Options**
3. In the next menu, select **Wayland**
4. Select **X11**, then **<OK>**
5. Use the right arrow key to select **<Finish>** and then **Reboot**

2 Set-up

The touch monitor is plug-and-play. Since the RPi 5 only has a single USB-C port, which is used for the power supply, the monitor needs to be connected to the RPi via both HDMI for data and USB for power. I used the HDMI to mini-HDMI cable that came with the RPi and the USB-C to USB-A cable that came with the monitor. The RPi did need to be rebooted after everything was plugged in for the touch screen to work properly.

2.2 GrandOrgue installation

From the GrandOrgue releases GitHub, download the `arm64.deb` architecture package. As of May 2026, the current release was `grandorgue_3.17.1-1_arm64.deb`. You can also download it directly in the terminal:

```
wget https://github.com/GrandOrgue/grandorgue/releases/download/3.17.1-1/grandorgue_3.17.1-
```

(Note: `grandorgue` requires **wxWidgets 3.2** and `grandorgue-wx30` requires **wxWidgets 3.0**. Raspberry Pi OS Bookworm ships with wxWidgets 3.2, so the standard `grandorgue` package above is the correct one.)

Install the software using either the downloaded installer or terminal:

```
sudo apt install ./grandorgue_3.17.1-1_arm64.deb
```

You might get a `Permission Denied` warning, but it can be ignored:

```
N: Download is performed unsandboxed as root as file '/home/username/grandorgue_3.17.1-1_ar
```

It is not a fatal error; the application should have installed fine.

Verify the installation by looking for it in the **Sound & Video** application menu. Launch the program. It should open with the demo organ available.

2.2.1 MIDI

Next, configure the MIDI hardware:

1. Plug in the Behringer UMC204HD USB audio interface to the RPi via USB and to the keyboard or organ using MIDI cables.

2 Set-up

2. Open GrandOrgue and go to File ▢ Preferences ▢ Audio. You should see the UMC204HD listed as an ALSA device. Select it, set the sample rate to 48000 Hz and start with 512 samples per buffer.
3. You can tune the buffer size down later if latency feels sluggish, or up if you hear crackle.
4. Go to File ▢ Preferences ▢ MIDI Devices. The UMC204HD's MIDI ports should appear as ALSA MIDI devices.
5. Enable the input port. Then load a sample set and use File ▢ MIDI Recorder or just play a key. GrandOrgue will prompt you to map MIDI channels to manuals the first time.

2.2.2 GrandOrgue configuration

Lastly, we need to map MIDI channels between GrandOrgue and the instrument.

1. Go to File ▢ Settings ▢ MIDI Devices
2. Under "MIDI output devices", make sure UMC204HD is selected. Click OK.
3. On the virtual pipe organ (VPO) console, right-click a virtual manual to map to.
4. In the box that opens, click "Listen for Event". Play a key on the keyboard you wish to map to that virtual organ manual.
5. Click OK.

To reduce audio lag:

1. File ▢ Settings ▢ Audio
2. Set the "Samples per buffer" to 256 or 128 (defaults to 512).

The lower the buffer size, the lower the latency. At 48,000 Hz sample rate, 128 samples per buffer supposedly gives about 2.7 milliseconds of latency, which is completely imperceptible. The tradeoff is that very low buffer sizes can cause audio crackle or dropouts if the RPi can't process audio fast enough, but the Pi 5 should be plenty powerful enough to handle 128 samples without trouble, especially with the demo sample set.

Even at 128, I still noticed a lag, but it was better than when I started. It is possible that some delay is "built in" as the result of the sample recording. This remains to be determined. I am accustomed to old-technology, non-realistic digital organs where the sound is instantaneous, where many pipe organs have a slight (fraction of a second) lag between the time a key is depressed and the sound is actually heard.

2 Set-up

2.2.3 Audio

By default, the VPO plays through the computer speakers. In this setup, this means the speakers on the touch screen monitor. Much better sound quality can be obtained with better speakers or headphones.

Bluetooth headphones may experience a fairly significant lag in audio. It did for me. Since my old keyboard did not have an Audio In jack, I could not channel the sound back to the keyboard speakers – which is just as well, because they are not very good either. The best (but not perfect) option was wired headphones plugged into the UNC204HD interface. The only headphones I owned had a 1/8" plug, so I had to use a 1/4"-1/8" adapter to connect them. There was a slight buzz, possibly resulting from the extra connection. It was annoying, but not bad enough to abandon the setup.

3 Customize

3.1 Edit and add your pages

Edit the qmd or md files in the `content` folder. qmd files can include code (R, Python, Julia) and lots of Quarto markdown bells and whistles (like call-outs, cross-references, auto-citations and much more).

Each page should start with

```
---  
title: your title  
---
```

and the first header will be the 2nd level, so `##`. Note, there are situations where you leave off

```
---  
title: your title  
---
```

and start the qmd file with a level header `#`, but if using the default title yaml (in the `---` fence) is a good habit since it makes it easy for Quarto convert your qmd file to other formats (like into a presentation).

3.2 Add your pages the project

- Add the files to `_quarto.yml`

4 Customization

Using different organ sample sets

The default sample set is great to get started with, but it's rather small. The beauty of virtual pipe organ (VPO) software is the ability and flexibility to use any number of sample sets. Many sample sets of various sizes exist, from small single-division organs to large many-division instruments. One constraint is disk storage: one needs to have enough space to download and store the sample set. The primary constraint with choosing samples to use, however, is one's memory: the sample set must fit in the computer's RAM to work properly. Thus, my constraint was 16 GB RAM that came on the Raspberry Pi 5. What this means in practice:

- **Under 8 GB:** loads comfortably, plenty of headroom
- **8–14 GB:** should work fine, some breathing room for OS
- **14–15 GB** getting risky; the OS and GrandOrgue itself need some RAM too, so you don't want to use all 16 GB for the sample set
- **Over 15 GB:** will not work; not enough memory for GrandOrgue and the OS

Since I only had one keyboard with no pedalboard, it made sense to consider single-manual samples. I could use larger ones, but I could only play one manual. The workaround would be mapping the Great division to my keyboard and relying on couplers to play other manuals from the Great manual.

4.1 Bureå Funeral Chapel

The first organ I tried was the Bureå Funeral Chapel from Lars Virtual Pipe Organs. This is a small chamber organ with one manual, pedals, and 9 stops. All of the manual stops are split stops with separate drawknobs for bass (first two octaves) and division. This provides greater flexibility on a small instrument, allowing the organist to use different combinations of stops on one manual instead of across two manuals.

4 Customization

First, I downloaded the .rar file. I needed to install the “unrar” utility the first time in order to extract this:

```
sudo apt install unrar
```

Then I unpacked it into the GrandOrgue/Organs/ directory:

```
unrar x Burea_Funeral_Chapel.rar /home/username/GrandOrgue/Organs/Burea_Funeral_Chapel/
```

Open GrandOrgue, then got to File □ Open, find and select the desired organ sample, and click “Open”. If everything worked, the new organ should load and be ready for use.

5 Rendering

The repo includes a GitHub Action that will render (build) the website automatically when you make changes to the files. It will be pushed to the `gh-pages` branch.

But when you are developing your content, you will want to render it locally.

5.1 Step 1. Make sure you have a recent RStudio

Have you updated RStudio since about August 2022? No? Then update to a newer version of RStudio. In general, you want to keep RStudio updated and it is required to have a recent version to use Quarto.

5.2 Step 2. Clone and create RStudio project

First, clone the repo onto your local computer. How? You can click `File > New Project` and then select “Version Control”. Paste in the url of the repository. That will clone the repo on to your local computer. When you make changes, you will need to push those up.

5.3 Step 3. Render within RStudio

RStudio will recognize that this is a Quarto project by the presence of the `_quarto.yml` file and will see the “Build” tab. Click the “Render website” button to render to the `_site` folder.

Previewing: You can either click `index.html` in the `_site` folder and specify “preview in browser” or set up RStudio to preview to the viewer panel. To do the latter, go to `Tools > Global Options > R Markdown`. Then select “Show output preview in: Viewer panel”.

6 References

Quarto has powerful references functionality. You can easily insert citations from Zotero libraries that you maintain in the cloud (on Zotero). This allows the whole team to update the library and you can sync up to that library. Read about this on the Quarto documentation on citations. Google youtube videos on this also to see it in action.

Add a `.bib` file in to your project or add a linked Zotero library via RStudio in Visual mode with Tools > Project Options... > R Markdown > select custom libraries from the Zotero dropdown.

Then you can type @ and you will see a dropdown of the references in your libraries. You can then select the ones to add. If you don't see the one you need, you can paste in the DOI and it will be added to your references file (with all the info). The references will be added to your references section of your book automatically.

See the `references.qmd` file for how to include the references.

- @ansley1981 will produce Ansley and Davis (1981)
- [@ansley1981] will produce (Ansley and Davis 1981).

References

Ansley, H. L. H., and C. D. Davis. 1981. *Migration and Standing Stock of Fishes Associated with Artificial and Natural Reefs on Georgia's Outer Continental Shelf*. Brunswick, Georgia, USA.